

Track B Chapter — Temporal Head Training, Fusion, and Evaluation

This chapter presents Track B as a self-contained unit for a thesis: conceptual framing, mathematical underpinnings, data assumptions, and a fully reproducible guide to training and evaluating the temporal head that consumes Track A Stage B outputs. The goal is to allow a reader to recreate experiments, audit design choices, and understand how priors (hotspots, CLIP) interact with multi-task objectives (next-active, noun/verb semantics, time-to-contact).

Table of Contents

0. Reading Guide and Intended Audience
1. Thesis Motivation and Scope
2. Problem Statement and Notation
3. Pipeline Context (Track A → Track B → Track C)
4. Data Assets, Paths, and Layout
5. Input Manifests (from Track A Stage B)
6. Candidate Semantics and Label Availability
7. Model Overview (Backbone → Projector → Fusion → Head)
8. Tokenization Theory (Grid Tokens, Temporal Windows)
9. Frame-Guided Temporal Pooling (FGTP) — Theory and Rationale
10. Dual Cross-Attention Fusion — Theory and Rationale
11. Prediction Heads — Next-Active, Noun, Verb, TTC (Reg + Bin)
12. Loss Functions, Weighting, and Optimization
13. TTC Regression vs TTC Binning — When and Why
14. Priors at Evaluation Time (Hotspot) — Theory and Implementation
15. CLIP Re-Ranking — Theory, Prompts, and Blending
16. Dataset Loader — Parsing, Validation, and Caching
17. Tokenization Implementation (ResNet18 vs VideoMAE Ego)
18. Pre-Extracted Tokens — Rationale and Workflow
19. Track B Configuration Files (trackB.yaml, trackB_videomae_ego.yaml)
20. TrainConfig Deep Dive (trackB_train_loader.py)
21. Training Pipeline (Main Mode) — Step-by-Step
22. Demo Mode (Loader and Standalone Variants)
23. Validation During Training — Metrics and Early Stopping
24. Checkpointing, Summaries, and Naming Conventions
25. Evaluation Pipeline (trackB_eval.py) — Step-by-Step
26. Metrics Definitions (Candidate-Level, Frame-Level, Top-5)
27. Outputs and File Schemas (Metrics, Predictions, Overlays)
28. Experiment Recipes (Baseline, Multi-Task, Priors, Tokens)
29. Performance, Resource, and Determinism Considerations
30. Reproducibility and Logging (RunLogger, Config Snapshots)
31. Troubleshooting and Failure Modes (Extensive)
32. Data Quality Checks and Manifest Authoring Tips

33. Ablations and Sensitivity Analyses (Fusion Depth, Priors, Bins)
 34. Integration with Track A (Manifests) and Track C (Pruning)
 35. VideoMAE Ego Configuration and Pretraining Notes
 36. Security, Safety, and Cleanup Guidelines
 37. Pseudo-Code Walkthroughs (Tokenization, Fusion, Loss)
 38. Extended Parameter Reference (Tokenizer, Dataset, Train, Eval)
 39. File Path Cheat-Sheet and Naming Standards
 40. Example Records (Manifest, Predictions, Metrics)
 41. CLI and PowerShell Command Library
 42. Glossary of Terms and Abbreviations
 43. Closing Notes for Thesis Integration
-

0. Reading Guide and Intended Audience

- Audience: thesis readers, reviewers, and practitioners who need both conceptual grounding and executable steps.
 - Style: prose for narrative; bullets for operations; code blocks for commands.
 - Scope: Track B only, assuming Track A Stage B has produced head manifests and crops. Track C is referenced for context but not detailed here.
 - Paths: all paths are relative to repository root unless otherwise specified.
-

1. Thesis Motivation and Scope

- Objective: learn a temporal head that predicts the next-active object (and semantics) using ego-centric frames and proposals generated in Track A.
 - Motivation: separate spatial proposal quality (Track A) from temporal/semantic reasoning (Track B) to isolate design effects.
 - Scope boundaries:
 - Included: dataset parsing, tokenization, fusion, heads, losses, training, evaluation, priors, CLIP re-ranking, outputs.
 - Excluded: proposal generation (Track A), pruning (Track C), external datasets beyond provided manifests.
 - Thesis contribution: demonstrate how a lightweight fusion head with optional priors can deliver competitive $N/N_v/N+\delta$ metrics while remaining reproducible and resource-aware.
-

2. Problem Statement and Notation

- Inputs:
 - Frame $I_{u,f}$ for uid u at frame index f .
 - Candidate set $C_{u,f} = \{b_i\}$ with boxes $b_i = (x1, y1, x2, y2)$ plus optional semantics.
- Outputs per candidate:
 - Next-active logits $p_{next}(b_i)$ (binary or multi-class).
 - Optional semantics: noun n_i , verb v_i , TTC regression t_i , TTC bin tb_i .
- Objectives:

- Classification loss on next-active (`is_positive` preferred; falls back to `cls`).
 - Optional CE on noun/verb when labels exist.
 - TTC regression (SmoothL1) or TTC bin CE when enabled.
 - Notation:
 - `K_c` : number of candidates in a frame (variable).
 - `T` : temporal window length (`time_len`).
 - `τ` : IoU threshold for frame-level semantic metrics (default 0.5).
 - `α_hotspot` : blend weight for hotspot priors; `α_clip` : blend weight for CLIP re-rank.
-

3. Pipeline Context (Track A → Track B → Track C)

- Track A Stage B produces `head_train.jsonl` / `head_val.jsonl` with proposals and semantics.
 - Track B trains a temporal head on these manifests, writing checkpoints and metrics to `local_extraction/runs/Track_B/`.
 - Track C pruning consumes the same manifests and a Track B checkpoint to study latency-accuracy trade-offs.
 - Auto-discovery: Track B scripts can discover the latest Stage B run if paths are not explicitly set.
-

4. Data Assets, Paths, and Layout

- Frames: `local_extraction/v2/extracted_frames/<uid>/<frame:07d>.jpg`
 - Stage B manifests (preferred): produced under `local_extraction/runs/Track_A/trackA_stageB_<timestamp>/`
 - Fallback manifests (optional): `local_extraction/v2/manifests/head_*_{clip|video}.json`
 - Track B outputs: `local_extraction/runs/Track_B/{checkpoints,metrics,predictions,overlays,plots,cache}/`
 - Configs:
 - Baseline: `local_extraction/configs/trackB.yaml`
 - VideoMAE Ego: `local_extraction/configs/trackB_videomae_ego.yaml`
 - Tokens (optional speedup): `tokens_root` pointing to pre-extracted ResNet18 or VideoMAE tokens.
-

5. Input Manifests (from Track A Stage B)

- Expected fields per record (grouped by frame):
 - `uid` : string
 - `frame` : int (or `frame_idx` , `frame_index` , `frame_name` , `image`)
 - `candidates` : list of candidate dicts or Stage B consolidated JSONL rows
 - Candidate keys: `x1,y1,x2,y2` , `is_positive` , `cls` (fallback), `verb_id` , `noun_id` , `ttc` , optional `ttc_bin`
- Flexible parsing: dataset accepts aliases (`boxes` , `objects` , `label` , `time_to_contact` , etc.).
- Stage B consolidated rows (`head_train.jsonl` format) are automatically regrouped per frame during parsing.
- TTC bins are computed if missing using thresholds (`0.5` , `1.0` , `2.0`) by default.

- Synthetic fallback: if no manifest exists and `synthetic_if_empty=True`, the dataset fabricates one candidate per UID (for smoke tests only).
-

6. Candidate Semantics and Label Availability

- Primary label: `is_positive` (next-active flag). If absent, `cls==1` is treated as positive.
 - Semantics (optional): `verb_id`, `noun_id`, `ttc`, `ttc_bin`.
 - Handling missing semantics:
 - Noun/verb losses are gated by `use_multi_task_labels`; vocabularies are inferred from observed IDs.
 - TTC bin head is activated only if `use_ttc_bins=True` and bins exist (or are computed).
 - Frame-level metrics assume at most one positive per frame; the first positive index is used when multiple are present.
-

7. Model Overview (Backbone → Projector → Fusion → Head)

- Backbone: ResNet18 (2D per-frame) or VideoMAE Ego (3D) to produce grid tokens.
 - Projector: linear layer mapping backbone channel dimension to `token_dim` (default 256).
 - Fusion: Frame-Guided Temporal Pooling (FGTP) + dual cross-attention layers operating on projected tokens.
 - Head: multi-branch predictor for next-active classification, optional noun/verb logits, TTC regression, and TTC bins.
 - Design intent: keep the head lightweight, disentangled from proposal quality, and compatible with pre-extracted tokens for speed.
-

8. Tokenization Theory (Grid Tokens, Temporal Windows)

- Spatial tokens: flatten the final backbone feature map into (N, C) where $N = H_f * W_f$.
 - Temporal tokens: sample a short window of frames ending at the target frame (`time_len`, `time_stride`), produce (T, N, C) .
 - Assumption: last frame is the anchor for next-active detection; temporal context provides motion cues for TTC and semantics.
 - ROI pooling later maps candidate boxes from pixel space onto the token grid for both image and video tokens.
-

9. Frame-Guided Temporal Pooling (FGTP) — Theory and Rationale

- FGTP projects temporal tokens onto the last-frame grid to align motion and appearance.
 - Each spatial location in the last frame serves as a query to pool corresponding trajectories across the temporal window.
 - Motivation: ego-centric motion can smear features; aligning to the anchor frame stabilizes candidate pooling.
 - Complexity: lightweight attention-like pooling rather than full spatiotemporal attention to keep latency low.
-

10. Dual Cross-Attention Fusion — Theory and Rationale

- After FGTP, dual cross-attention layers exchange information between image tokens and temporally pooled tokens.
 - Symmetric design: image attends to video, video attends to image, encouraging complementary cues (appearance vs motion).
 - Depth/head defaults (layers=2, heads=8, dim=256) target a sweet spot between expressiveness and speed.
 - Optional deeper stacks can be used in ablations (see Section 33).
-

11. Prediction Heads — Next-Active, Noun, Verb, TTC (Reg + Bin)

- Next-active head: classification over `num_classes` (at least binary).
 - Noun/verb heads: enabled when `use_multi_task_labels=True`; vocabularies inferred from manifest IDs and stored in checkpoints (`noun_id_list` , `verb_id_list`).
 - TTC regression head: outputs continuous TTC; always present for compatibility even when bins are used.
 - TTC bin head: enabled when `use_ttc_bins=True` or when a checkpoint contains bin weights.
 - All heads operate on ROI-pooled features for each candidate box.
-

12. Loss Functions, Weighting, and Optimization

- Next-active: CrossEntropy with optional label smoothing (`training.label_smoothing`).
 - TTC regression: SmoothL1 (implemented via L1 with masking on candidates).
 - TTC bin: CrossEntropy over `num_ttc_bins` .
 - Noun/verb: CrossEntropy applied only on positive candidates (mask where `labels==1`).
 - Weights:
 - `loss_w_next` , `loss_w_noun` , `loss_w_verb` , `loss_w_ttc` from `multi_task.loss_weights` .
 - Training script multiplies TTC regression weight by 0.1 internally (`TTC_LOSS_WEIGHT = cfg_weight * 0.1`).
 - Optimization: Adam over projector, fusion, head; cosine LR schedule with warmup (`warmup_epochs`).
 - Mixed precision: optional (`training.amp`), disabled by default for stability.
-

13. TTC Regression vs TTC Binning — When and Why

- Regression (`ttc_mode=reg`): precise TTC predictions; metrics use MAE on denormalized seconds.
 - Binning (`ttc_mode=bin`): discretizes TTC into bins (default thresholds 0.5, 1.0, 2.0) and uses CE loss.
 - Eval flexibility: `trackB_eval.py` supports `--ttc_mode reg|binned` ; bin mode can derive bins from regression if bin logits are absent.
 - Empirical consideration: binning can stabilize training under label noise; regression may capture fine-grained timing when labels are dense.
-

14. Priors at Evaluation Time (Hotspot) — Theory and Implementation

- Hotspot priors encode prior likelihoods for noun/verb pairs.
- JSON format:

```
{"pairs": {"noun_id,verb_id": score}, "default": 0.0}
```

- Scoring: $\text{score_final} = (1 - \alpha) * \text{base} + \alpha * \text{prior}$, where `base` is next-active prob.
- Scope: applied only at evaluation; does not change training or checkpoint contents.
- Controls: `evaluation.hotspot_priors.enabled`, `path`, `alpha` in `trackB.yaml`; can be toggled in `EvalConfig`.

15. CLIP Re-Ranking — Theory, Prompts, and Blending

- Purpose: refine next-active scores using text-image similarity on predicted nouns.
- Prompts: `a photo of {noun_label}` derived from STA annotation JSON (`paths.org_annotations`).
- Scoring: cosine similarity mapped to `[0,1]` via $0.5 * (\text{sim} + 1)$.
- Blending: $\text{score} = (1 - \text{clip_weight}) * \text{score} + \text{clip_weight} * \text{score_clip}$.
- Requirements: `clip` package, noun label file, and predicted noun IDs.
- Scope: evaluation-only; independent from training loss.

16. Dataset Loader — Parsing, Validation, and Caching

- Source: `local_extraction/trackB/trackB_dataset.py`.
- Responsibilities:
 - Load manifest (JSON array or JSONL) and regroup Stage B consolidated rows.
 - Discover manifests automatically if `manifest_path` not provided.
 - Convert candidate schemas to normalized fields (`bbox`, `cls`, `is_positive`, `verb_id`, `noun_id`, `ttc`, `ttc_bin`).
 - Compute TTC statistics (mean/std/min/max) and normalize TTC if enabled.
 - Cache invalid records to `runs/Track_B/cache/invalid_records.csv`.
 - Fabricate synthetic records if allowed and manifest is empty.
- Frame resolution: `_resolve_frame_path` prefers absolute `image`, else derives from `frames_root/uid`, `frame_name`, `index`, then falls back to last frame.
- Candidate limiting: random subsample if `candidate_limit` exceeded per sample.
- DataLoader workers: must stay `0` because tokenizer/backbone objects live inside the dataset.

17. Tokenization Implementation (ResNet18 vs VideoMAE Ego)

- Config: `TokenizerConfig` with `video_backbone` set via YAML or CLI-config override.
- ResNet18 path:
 - Uses torchvision ResNet18, layer4 output (C=512), frozen by default.
 - Transforms: resize/crop to `img_size` (default 224), mean/std normalization.

- VideoMAE Ego path:
 - Wrapper over VideoMAE encoder with temporal tubelets; outputs spatial tokens reshaped to grid.
 - Supports loading pretrained weights (`model.tokenizer.videomae.weights_path`) or HF names via `trackB_videomae_ego.yaml` .
 - Frozen encoder by default (`videomae_freeze_encoder=True`).
 - Temporal sampling: `sample_window_ending_at` builds a window of length `time_len` with stride `time_stride` ending at anchor frame.
 - ROI pooling: `roi_pool_tokens_mean` averages token grid values inside the projected ROI for both image and video tokens.
-

18. Pre-Extracted Tokens — Rationale and Workflow

- Motivation: avoid repeated backbone passes; ~120× speedup claimed for ResNet18 tokens.
 - Training detection: if `data.tokens_root` is set, training script attempts to auto-detect token dimension (ResNet18 vs VideoMAE).
 - Token format expectations:
 - ResNet18 tokens: dict with `tokens` (N,C) in each `.pt` file.
 - VideoMAE tokens: dict with `img_tokens` (N,C) and optionally clip tokens.
 - Usage: set `data.tokens_root` in YAML (relative to `local_extraction` or absolute).
 - Evaluation: resolves `tokens_root` from checkpoint `train_config` to ensure consistent backbone assumptions.
-

19. Track B Configuration Files (trackB.yaml, trackB_videomae_ego.yaml)

- `trackB.yaml` (baseline):
 - Video backbone: `resnet18` by default, frozen.
 - Projector: 512→256.
 - Fusion: dim=256, layers=2, heads=8, dropout=0.1.
 - Head: dim=256, hidden=256, num_classes=2.
 - Multi-task: noun/verb/TTC enabled by default; TTC mode `reg` ; thresholds [0.5, 1.0, 2.0].
 - Training: 20 epochs, batch 8, lr 1e-3, warmup 1, label smoothing 0.05, weight decay 0.01, candidate_limit 16, normalize_ttc true.
 - Early stopping: patience 5 on `mAP` , mode `max` .
 - Evaluation: batch 8, overlays enabled, hotspot priors enabled (`hotspot_priors_train_logodds_min10.json`), CLIP re-rank enabled.
 - Data: `stageB_run` null (auto-detect), manifests null (auto), tokens_root null.
 - `trackB_videomae_ego.yaml` (ego VideoMAE):
 - Inherits baseline; switches `video_backbone` to `videomae_ego` , `time_len` to 16.
 - Uses VideoMAE pretrained weights (`videomae_local_part/.../videomae_ego_scratch_last.pt` or HF model).
 - Projector input 768→256; lr lowered to 5e-4; warmup 2 epochs.
 - Tokens_root example: `videomae_trackB_tokens_scratch/tokens` .
-

20. TrainConfig Deep Dive (trackB_train_loader.py)

- Mode and demo: `mode` (`main` / `demo`), `demo_steps` , `demo_variant` (`loader` / `standalone`), `demo_use_real_labels` .
 - Hyperparameters: `epochs` , `batch_size` , `lr` , `min_lr` , `warmup_epochs` , `candidate_limit` , `label_smoothing` , `normalize_ttc` , `amp` .
 - Checkpointing: `save_epoch_checkpoints` , `save_best_checkpoint` .
 - Evaluation cadence: `eval_every` .
 - Early stopping: `monitor_metric` , `greater_is_better` (auto-set for `ttc_mae`), `early_stopping_patience` .
 - Data: `train_manifest` , `val_manifest` , `stageB_run` (for auto-discovery), `tokens_root` .
 - Multi-task: `use_multi_task_labels` , `use_ttc_bins` .
 - Loss weights: `loss_w_next` , `loss_w_noun` , `loss_w_verb` , `loss_w_ttc` (note: TTC multiplied by 0.1 internally).
 - CLI overrides: `--config` , `--demo` , `--epochs` , `--batch_size` , `--lr` .
-

21. Training Pipeline (Main Mode) — Step-by-Step

1. Load YAML (`--config` if provided); log backbone choice.
 2. Set tokenizer config globally (for shared `TokenizerConfig`).
 3. Resolve frames root (`local_extraction/v2/extracted_frames`).
 4. Discover Stage B run (latest `trackA_stageB_*`) unless `stageB_run` is provided.
 5. Resolve manifests: prefer Stage B `head_train/head_val` ; fall back to `local_extraction/v2/manifests` .
 6. Resolve `tokens_root` if configured; auto-detect token dimension to set projector input.
 7. Build training dataset (`TrackBDataset`) with candidate limit and TTC normalization.
 8. Infer vocab sizes from parsed data:
 - Next-active classes: `max(cls) + 1` , minimum 2.
 - Noun/verb IDs collected and mapped to contiguous vocab indices.
 - TTC bins inferred if `use_ttc_bins=True` .
 9. Instantiate projector, fusion, head; move to device (`TokenizerConfig.device` : auto→CUDA/CPU).
 10. Build DataLoader (workers=0).
 11. Optionally build val loader (auto-discovered val manifest).
 12. Training loop per epoch:
 - For each batch:
 - Tokenize already done in dataset; project image/video tokens.
 - Fusion (FGTP + cross-attention).
 - ROI-pool per candidate; pad to max candidates in batch; build masks.
 - Forward head; compute losses (next-active, TTC reg/bin, noun/verb).
 - Apply loss weights; scale with AMP if enabled.
 - Cosine LR schedule with warmup.
 - Backprop, optimizer step, scaler update.
 - Log epoch averages.
 - Save epoch checkpoint if enabled.
 - Run validation every `eval_every` epochs; track best metric; save best checkpoint + summary JSON.
 13. Final checkpoint saved regardless of mode; preview batch metrics logged.
-

22. Demo Mode (Loader and Standalone Variants)

- Purpose: quick functional sanity without full training.
 - Loader variant: iterates a few DataLoader batches, computing losses and stepping optimizer.
 - Standalone variant: samples frames directly, with synthetic boxes if no manifest; useful when manifests are absent.
 - Controls: `--demo` CLI flag or set `demo.enabled` in YAML; `demo_steps`, `demo_variant`, `demo_use_real_labels`.
 - Outputs: no saved checkpoint by default; intended for shape/loss verification.
-

23. Validation During Training — Metrics and Early Stopping

- `_evaluate_loader` computes:
 - Candidate-level accuracy (binary or multi-class) and mAP (binary AP for class 1).
 - TTC MAE on denormalized seconds.
 - Candidate counts and class distribution.
 - Early stopping: compares monitored metric (`monitor_metric`) with best; patience `early_stopping_patience`; `greater_is_better` automatically set to `False` for `ttc_mae`.
 - Best checkpoint: `trackB_best_<metric>_<score>_<ts>.pt` plus alias `trackB_best.pt`; summary JSON mirrors metrics and config.
-

24. Checkpointing, Summaries, and Naming Conventions

- Directory: `local_extraction/runs/Track_B/checkpoints/`.
 - Files:
 - Epoch checkpoints: `trackB_epoch<E>_<timestamp>.pt` when enabled.
 - Best checkpoint: `trackB_best_<metric>_<score>_<ts>.pt` + `trackB_best.pt` alias.
 - Final checkpoint: `trackB_final_<timestamp>.pt` always saved.
 - Summaries: `trackB_best_<metric>_<score>_<ts>_summary.json`, `trackB_best_summary.json`.
 - Contents: `projector`, `fusion`, `head` state_dicts; `noun_id_list`, `verb_id_list`; serialized `train_config` (scalar-friendly).
 - Notes: projector input dim and detected backbone are stored to disambiguate pre-extracted tokens.
-

25. Evaluation Pipeline (trackB_eval.py) — Step-by-Step

1. Load EvalConfig from YAML (paths, batch size, priors, CLIP).
2. Resolve Stage B run (latest if unspecified) and val manifest (`head_val` preferred, then fallbacks).
3. Resolve checkpoint: CLI `--checkpoint` or latest `trackB_final_*.pt` / `trackB_best.pt`.
4. Load checkpoint; infer class counts and projector input dim from weights; rebuild projector, fusion, head.
5. Build TokenizerConfig with tokens_root/backbone inferred from checkpoint.
6. Build `TrackBDataset` for validation with candidate limiting and TTC normalization.
7. Optional debug: count empty candidates and missing frame paths; write missing UID list to cache.
8. Iterate samples (workers=0):

- Project and fuse tokens; ROI-pool per candidate.
 - Forward head; get logits, TTC reg, optional noun/verb/bin logits.
 - Apply hotspot priors (if enabled) to next-active scores.
 - Apply CLIP re-rank (if enabled) using noun predictions and text prompts.
 - Collect predictions, scores, TTC, semantics, and per-frame metrics.
 - Draw overlays for top-K candidates if `save_overlays=True`.
9. Aggregate metrics (candidate-level and frame-level, plus top-5 variants).
 10. Write outputs: metrics JSON + summary, predictions CSV/JSONL, overlays.
 11. Log via RunLogger if available.
-

26. Metrics Definitions (Candidate-Level, Frame-Level, Top-5)

- Candidate-level:
 - `accuracy` : next-active accuracy (binary or multi-class).
 - `mAP` : binary AP for positive class or macro over classes.
 - `ttc_mae_seconds` : MAE on denormalized TTC.
 - Frame-level (top-1): computed on highest-scoring candidate whose box IoU $\geq \tau$ with GT positive box:
 - `N_mAP` : noun match.
 - `Nv_mAP` : noun + verb match.
 - `N_delta_mAP` : noun + TTC-bin match (pred bin from head or regression).
 - `All_mAP` : noun + verb + TTC-bin match.
 - Frame-level (top-5 hit-rate): fraction of frames with a correct match within top-5 candidates by score for each metric variant.
 - Top-5 candidate-level AP: AP computed only over top-5 candidates per frame for N/Nv/N+ δ /All labels.
 - Per-class breakdowns: accuracy per noun/verb ID when semantics exist.
-

27. Outputs and File Schemas (Metrics, Predictions, Overlays)

- Metrics: `local_extraction/runs/Track_B/metrics/metrics_val_<ts>.json` and `_summary.json` (includes eval_config).
 - Predictions:
 - CSV: `predictions_val_<ts>.csv` with columns `uid, frame_path, cand_idx, x1, y1, x2, y2, label, pred_label, prob_pos, score_final, score_hotspot, score_clip, ttc_pred_s, ttc_gt_s, ttc_error_s, gt_noun_id, pred_noun_id, gt_verb_id, pred_verb_id, gt_ttc_bin, pred_ttc_bin`.
 - JSONL: same fields per line.
 - Overlays: JPEGs under `overlays/val/`, named `<uid>_<frame>.jpg`, with drawn boxes and TTC text.
 - Cache: invalid records CSV, missing UID list (debug) under `runs/Track_B/cache/`.
-

28. Experiment Recipes (Baseline, Multi-Task, Priors, Tokens)

- Baseline single-task (next-active + TTC reg):
 - Set `multi_task.enabled=False`, `multi_task.ttc_mode="reg"`, `evaluation.hotspot_priors.enabled=False`,

```
evaluation.clip_rerank.enabled=False .
```

- Train/Eval as usual.
- Full multi-task + bins:
 - `multi_task.enabled=True` , `multi_task.predict_noun=True` , `predict_verb=True` , `use_ttc_bins=True` (set in TrainConfig).
 - Eval with `--ttc_mode binned` ; priors/CLIP optional.
- Priors + CLIP eval only:
 - Train with multi-task on.
 - Eval: `use_hotspot_priors=True` , `hotspot_alpha=0.3` ; `use_clip_rerank=True` , `clip_weight=0.3` .
- Pre-extracted tokens:
 - Generate tokens (external script, not included here).
 - Set `data.tokens_root` to token directory; train/eval will skip backbone builds.
- VideoMAE ego:
 - Use `--config trackB_videomae_ego` ; ensure weights exist; adjust batch size if memory constrained.

29. Performance, Resource, and Determinism Considerations

- GPU vs CPU: training/eval run on CPU but are slower; GPU preferred.
- DataLoader workers: keep `0` to avoid pickling tokenizer/backbone.
- Tokenization cost: dominant when not using pre-extracted tokens; `tokens_root` mitigates.
- Memory: candidate padding per batch scales with max candidates; set `candidate_limit` to control.
- Determinism:
 - File iteration sorted in dataset; random subsampling of candidates uses Python `random` seeded in dataset init.
 - No global torch seed set beyond `torch.manual_seed(0)` in train loader.
- LR schedule: cosine with warmup; monitor for over-decay on very short runs.

30. Reproducibility and Logging (RunLogger, Config Snapshots)

- RunLogger: both train and eval attempt to log configs, metrics, artifacts; failures are non-fatal.
- Config provenance:
 - Train: `_config_to_dict` snapshot embedded in checkpoints and best summaries.
 - Eval: EvalConfig serialized into `_summary.json` .
- Naming: include mode/backbone/priors in run notes for clarity.
- Artifact retention: keep `trackB_best.pt` , associated summary JSON, and metrics/predictions for thesis figures.

31. Troubleshooting and Failure Modes (Extensive)

- **No candidates parsed:** manifest empty or schema mismatch; run `trackB_manifest_validator.py` (if present) or inspect `invalid_records.csv` .
- **Missing frames:** `frames_root` may differ; debug output logs missing UID directories; verify `local_extraction/v2/extracted_frames` .

- **Zero positives in val:** AP/mAP becomes ill-defined; check that `is_positive` is set and manifest aligns with frames.
 - **Cuda OOM:** reduce `batch_size`, `candidate_limit`, fusion layers; disable AMP if unstable.
 - **Priors/CLIP have no effect:** ensure `use_hotspot_priors` / `use_clip_rerank` True, paths valid, and noun/verb heads enabled.
 - **ttc_mode confusion:** training may use regression; eval can still bin regression outputs; set `--ttc_mode` explicitly.
 - **Slow epoch:** use `tokens_root`; shorten `time_len`; lower fusion depth; limit candidates.
 - **Overlays missing:** set `save_overlays=True`; ensure frames exist; reduce `topk_overlay`.
 - **Best checkpoint missing:** monitor metric NaN or invalid; ensure `save_best_checkpoint=True` and val loader not empty.
 - **Class mismatch:** noun/verb vocab inferred from training manifest; keep consistent IDs across train/val.
 - **Synthetic samples showing up:** indicates manifests were empty; set correct paths or disable `synthetic_if_empty`.
 - **VideoMAE weights not found:** check `videomae_local_part/...` path or HF model name in config.
 - **Token detection wrong:** ensure `tokens_root` files include expected keys (`tokens` or `img_tokens`).
-

32. Data Quality Checks and Manifest Authoring Tips

- Ensure each frame has at least one candidate with `is_positive=1` when semantics exist; otherwise metrics will undercount.
 - Maintain consistent taxonomy IDs for noun/verb across train and val.
 - If generating manifests manually, keep bounding boxes in pixel coordinates matching frame resolution.
 - Include TTC where available; dataset normalizes across entire parsed set.
 - Use Stage B outputs directly when possible to avoid schema drift.
-

33. Ablations and Sensitivity Analyses (Fusion Depth, Priors, Bins)

- Fusion depth/layer count: try `{2,3,4}` layers, heads `{4,8}`, dim `{192,256,320}`; monitor mAP vs runtime.
 - Candidate_limit: sweep `{4,8,16,32}` to study effect on accuracy and latency.
 - TTC mode: compare `reg` vs `bin` on `ttc_mae_seconds` and `N_delta_mAP`.
 - Multi-task toggles: on/off noun+verb to see impact on N/Nv/N+ δ metrics.
 - Priors: vary `hotspot_alpha` `{0.0,0.2,0.3,0.5}`; CLIP `clip_weight` similarly; evaluate on same checkpoint.
 - Tokens_root vs on-the-fly: measure wall-clock speed and accuracy parity to justify preprocessing.
-

34. Integration with Track A (Manifests) and Track C (Pruning)

- Track A: auto-discovery via `latest_stageB_run` under `local_extraction/runs/Track_A/`; Stage B manifests take priority over `v2/manifests`.
 - Track C: consumes Track B checkpoint and the same Stage B manifests; Track B need not change for pruning experiments.
 - Hand-off best checkpoint: `trackB_best.pt` is the standard input for Track C comparisons.
-

35. VideoMAE Ego Configuration and Pretraining Notes

- Config: `local_extraction/configs/trackB_videomae_ego.yaml` .
- Pretraining requirement: run VideoMAE pretraining script (not part of this chapter) to produce `videomae_ego_scratch_last.pt` or use HF pretrained names.
- Temporal settings: `time_len=16` , tubelet size 2, patch 16.
- Projector in-dim: 768 for base, 1024 for large, 1280 for huge.
- Training hyperparameters adjusted: lr 5e-4, warmup 2 epochs.
- Tokens_root: use generated VideoMAE tokens for speed; set path accordingly.

36. Security, Safety, and Cleanup Guidelines

- No network calls; all paths local.
- Runs are timestamped; existing checkpoints are not overwritten (best alias is duplicated).
- Delete old `runs/Track_B/{checkpoints,metrics,predictions,overlays}` manually to reclaim space.
- Avoid altering manifests in place; copy before edits to preserve provenance.

37. Pseudo-Code Walkthroughs (Tokenization, Fusion, Loss)

Tokenization (simplified)

```
frame_path = resolve(uid, frame_name)
window = sample_window_ending_at(frame_path, time_len, time_stride)
img_tokens = image_grid_tokens(frame_path, backbone, transform) # (N,C)
vid_tokens = video_grid_tokens(window, backbone, transform)      # (T,N,C)
```

ROI Pooling + Fusion

```
img_proj = projector(img_tokens)      # (N,dim)
vid_proj = projector(vid_tokens)      # (T,N,dim)
fused_img, fused_vid = fusion(img_proj[None], vid_proj[None])
roi_feats = [roi_pool_tokens_mean(hw, fused_img[0], box, image_size) for box
in bboxes]
```

Loss (per batch)

```
logits, noun_logits, verb_logits, ttc_reg, ttc_bin_logits =
head(roi_feats_img, roi_feats_vid)
loss_next = CE(logits, labels)
loss_ttc = SmoothL1(ttc_reg, ttc_targets)
```

```

if use_bins: loss_ttc = CE(ttc_bin_logits, ttc_bin_targets)
if multi_task and positives: loss_noun = CE(noun_logits[pos], noun_ids[pos]);
loss_verb = CE(verb_logits[pos], verb_ids[pos])
loss = w_next*loss_next + w_ttc*loss_ttc + w_noun*loss_noun + w_verb*loss_verb

```

38. Extended Parameter Reference (Tokenizer, Dataset, Train, Eval)

TokenizerConfig (from YAML defaults)

- `video_backbone` : `resnet18` or `videomae_ego`
- `img_size` : 224 (baseline)
- `time_len` : 8 (baseline), 16 (VideoMAE config)
- `time_stride` : 2
- `device` : `auto` → CUDA if available, else CPU
- `use_half` : False (optional)
- VideoMAE-specific: `videomae.weights_path` , `patch_size` , `tubelet_size` , `embed_dim` , `depth` , `num_heads` , `freeze_encoder`

TrackBDataset

- `frames_root` : `local_extraction/v2/extracted_frames`
- `manifests_root` : Stage B run or `local_extraction/v2/manifests`
- `manifest_path` : explicit or discovered (`head_train*.json[1]`)
- `tokenizer_cfg` : TokenizerConfig instance
- `time_len` : optional override (sets `tokenizer_cfg.time_len`)
- `candidate_limit` : default 32 (TrainConfig uses 16)
- `normalize_ttc` : True/False
- `synthetic_if_empty` : True/False
- `cache_dir` : default `runs/Track_B/cache`
- `tokens_root` : None or path to pre-extracted tokens

TrainConfig (trackB_train_loader.py)

- `mode` : `main` / `demo`
- `demo_steps` , `demo_variant` , `demo_use_real_labels`
- `epochs` , `batch_size` , `lr` , `min_lr` , `warmup_epochs`
- `candidate_limit` , `label_smoothing` , `normalize_ttc` , `amp`
- `save_epoch_checkpoints` , `save_best_checkpoint`
- `eval_every` , `early_stopping_patience` , `monitor_metric` , `greater_is_better`
- `val_manifest` , `train_manifest` , `stageB_run` , `tokens_root`
- `use_multi_task_labels` , `use_ttc_bins`
- `loss_w_next` , `loss_w_noun` , `loss_w_verb` , `loss_w_ttc`

EvalConfig (trackB_eval.py)

- Paths: `frames_root` , `manifests_root` , `trackA_runs_root`
- Manifests: `val_manifest` , `stageB_run`

- Checkpoint: `checkpoint_path` (else best/final auto)
- Model: `token_dim`, `num_classes`, `fusion_layers`
- Data: `batch_size`, `candidate_limit`, `normalize_ttc`
- Outputs: `topk_overlay`, `save_overlays`
- Multi-task eval: `ttc_mode`, `iou_thresh`
- Priors: `use_hotspot_priors`, `hotspot_prior_path`, `hotspot_alpha`
- CLIP: `use_clip_rerank`, `clip_weight`, `clip_model`, `noun_label_path`

39. File Path Cheat-Sheet and Naming Standards

- Frames root: `local_extraction/v2/extracted_frames/`
- Stage B run (latest): `local_extraction/runs/Track_A/trackA_stageB_*`
- Track B outputs: `local_extraction/runs/Track_B/`
 - Checkpoints: `checkpoints/trackB_{epoch|best|final}*.pt`
 - Metrics: `metrics/metrics_val_*.json` and `_summary.json`
 - Predictions: `predictions/predictions_val_*.jsonl`
 - Overlays: `overlays/val/*.jpg`
 - Cache: `cache/invalid_records.csv`, `missing_val_uids.txt`
- Configs: `local_extraction/configs/trackB.yaml`,
`local_extraction/configs/trackB_videomae_ego.yaml`
- Tokens (optional): set `data.tokens_root` relative to `local_extraction` unless absolute.

40. Example Records (Manifest, Predictions, Metrics)

Manifest record (Stage B JSONL regrouped)

```
{
  "uid": "03abc",
  "frame": 120,
  "candidates": [

    {"x1":123.4,"y1":56.7,"x2":200.1,"y2":180.0,"is_positive":1,"verb_id":7,"noun_id":42,"ttc":0.83},

    {"x1":10.0,"y1":10.0,"x2":40.0,"y2":50.0,"is_positive":0,"verb_id":-1,"noun_id":-1,"ttc":2.5}
  ]
}
```

Prediction row (CSV/JSONL)

```
{
  "uid":"03abc",
```

```
"frame_path": "local_extraction/v2/extracted_frames/03abc/0000120.jpg",
"cand_idx": 0,
"x1": 123.4, "y1": 56.7, "x2": 200.1, "y2": 180.0,
"label": 1, "pred_label": 1,
"prob_pos": 0.78, "score_final": 0.81, "score_hotspot": 0.70, "score_clip": 0.85,
"ttc_pred_s": 0.92, "ttc_gt_s": 0.83, "ttc_error_s": 0.09,
"gt_noun_id": 42, "pred_noun_id": 42,
"gt_verb_id": 7, "pred_verb_id": 7,
"gt_ttc_bin": 1, "pred_ttc_bin": 1
}
```

Metrics excerpt

```
{
  "accuracy": 0.71,
  "mAP": 0.64,
  "ttc_mae_seconds": 0.42,
  "N_mAP": 0.58,
  "Nv_mAP": 0.52,
  "N_delta_mAP": 0.47,
  "All_mAP": 0.45,
  "N_top5_acc": 0.74,
  "Nv_top5_acc": 0.68
}
```

41. CLI and PowerShell Command Library

Environment setup (PowerShell)

```
cd D:\Thesis\Ego4d-LiteSTA
.\local_extraction\.env\Scripts\Activate.ps1
```

Train (baseline config)

```
python local_extraction\trackB\trackB_train_loader.py
```

Train (VideoMAE ego config)

```
python local_extraction\trackB\trackB_train_loader.py --config  
trackB_videomae_ego
```

Train with quick overrides

```
python local_extraction\trackB\trackB_train_loader.py --epochs 5 --batch_size  
4 --lr 5e-4
```

Eval (regression TTC)

```
python local_extraction\trackB\trackB_eval.py --ttc_mode reg
```

Eval (binned TTC) with explicit manifest/checkpoint

```
python local_extraction\trackB\trackB_eval.py --checkpoint  
local_extraction/runs/Track_B/checkpoints/trackB_best.pt --val_manifest  
local_extraction/runs/Track_A/trackA_stageB_20250101_120000/head_val.jsonl --  
ttc_mode binned
```

Plot metrics (if plots script available)

```
python local_extraction\trackB\trackB_plots.py
```

42. Glossary of Terms and Abbreviations

- **FGTP**: Frame-Guided Temporal Pooling.
- **ROI**: Region of interest (candidate box).
- **TTC**: Time-to-contact (seconds).
- **TTC bin**: Discrete TTC interval based on thresholds.
- **Hotspot prior**: Prior score for noun/verb pair blended with next-active probability.
- **CLIP re-rank**: Blending next-active probability with CLIP text-image similarity on predicted noun.
- **N/Nv/N+ δ /All**: Frame-level metrics using noun, noun+verb, noun+TTC-bin, and noun+verb+TTC-bin.
- **tokens_root**: Directory of pre-extracted backbone tokens for speed.

- **Track A Stage B:** Pipeline that generates candidate crops and manifests consumed here.
- **Track C:** Pruning stage that compares latency/accuracy using Track B checkpoints.

43. Closing Notes for Thesis Integration

- Use this chapter as a methodological description: articulate how proposal recall (Track A) interacts with temporal fusion and priors (Track B) to produce final metrics.
 - When reporting results: specify `K` in Track A, TTC mode, multi-task toggles, priors/CLIP settings, backbone choice, and whether `tokens_root` was used.
 - Keep references to file paths explicit (e.g., `local_extraction/runs/Track_B/checkpoints/trackB_best.pt`) to support reproducibility.
-

Extended Narrative and Detailed Sections

The remainder of this chapter elaborates each component in depth, providing additional lines of explanation, rationale, and operational guidance suitable for inclusion in an academic document. Each subsection intentionally repeats key facts with expanded commentary to satisfy both thoroughness and the requested length.

A1. Detailed Motivation and Design Principles

- Separation of concerns: Track A handles detection/proposals; Track B isolates temporal reasoning to quantify its specific contribution.
- Lightweight fusion: avoids heavy video transformers to remain practical on commodity GPUs while still modeling motion.
- Modularity: projector, fusion, and head are distinct modules saved in checkpoints; they can be swapped or fine-tuned independently.
- Local-first: all scripts assume offline execution with fixed assets; no external downloads during training/eval.
- Auditability: summaries and metrics JSONs capture configuration snapshots to support reproducibility arguments in a thesis.

A2. Mathematical Framing of Next-Active Prediction

- Let X_t denote image tokens at time t (flattened grid), V the temporal token tensor over window T .
- FGTP computes $A = f_align(V, X_T)$ projecting temporal context onto anchor grid.
- Dual cross-attention updates (X_T, A) to (X_T', A') where attention weights capture complementary cues.
- Candidate ROI pooling produces $z_i = pool(X_T', b_i)$ and optionally $v_i = pool(A', b_i)$.
- Head maps concatenated or fused features to logits $g(z_i, v_i)$ for next-active and semantics.
- Loss aggregates per-candidate terms with weights; TTC regression may be normalized using dataset statistics.

A3. Dataset Robustness and Schema Flexibility

- Schema aliases reduce friction when manifests evolve; the parser tolerates `frame` , `frame_idx` , `frame_index` , `frame_name` , `image` .
- Candidate dicts accept `x1|xmin|left` , etc., to absorb different exporters.
- If `gt_box` exists and no candidates are present, the parser fabricates one positive candidate from that GT box.
- TTC binning is applied even when TTC is missing (bin from 0.0) to keep tensor shapes consistent.
- Invalid entries are logged rather than raising, supporting iterative cleanup.

A4. Tokenization Nuances

- ResNet18 tokens correspond to layer4 resolution (~7x7 when `img_size=224`); ROI pooling maps pixel boxes to this coarse grid.
- VideoMAE tokens: outputs (B, N+1, D); wrapper reshapes spatial tokens from last temporal slice to grid (ignoring CLS).
- Time stride: `time_stride=2` means window frames are sampled every other frame; adjust for faster/slower motion sensitivity.
- Device handling: TokenizerConfig chooses CUDA if available; VideoMAE can be memory-intensive—monitor VRAM.
- Half precision: optional via `runtime.use_half` ; not defaulted due to stability considerations.

A5. Fusion and Attention Choices

- FGTP vs full self-attention: FGTP is linear in tokens and less costly; dual cross-attention adds limited complexity.
- Heads=8 ensures sufficient angular resolution in attention without ballooning parameters.
- Dropout=0.1 provides mild regularization; ablate if overfitting is observed.
- Fusion layers stack identical blocks; residual connections maintain stability.

A6. Head Configuration and Vocab Inference

- Num_classes: at least 2 (binary). If manifest contains higher cls labels, head expands accordingly.
- Noun/verb vocabularies: collected from training manifests; stored in checkpoints to map local indices back to global taxonomy IDs.
- TTC bins: inferred from data if `use_ttc_bins=True` ; otherwise head omits bin logits.
- `ttc_mode` in EvalConfig: independent from training; evaluation can still bin regression outputs.

A7. Loss Masking and Positive Definition

- Positive mask: `is_positive` preferred over `cls` ; ensures backward compatibility with manifests that only provide class labels.
- Noun/verb losses: applied only on positives to avoid punishing negatives lacking semantics.
- TTC regression masking: uses candidate mask to avoid division by zero; SmoothL1 is averaged over valid candidates.
- Label smoothing: applied to next-active CE to mitigate noisy labels.

A8. Learning Rate Schedule and Warmup

- Warmup linear ramp from 0 to `lr` over `warmup_epochs * steps_per_epoch` .

- Cosine decay thereafter to `min_lr` ; ensures smooth convergence even for short runs.
- Recommended to keep `epochs` modest (e.g., 20) when using small datasets to prevent overfitting.

A9. Candidate Padding Strategy

- Variable candidates per frame are padded to `Nc_max` within a batch; mask identifies valid entries.
- Losses and TTC regression use mask to ignore padded slots.
- `Candidate_limit` enforces upper bound before padding to control tensor sizes.

A10. Val Loader Construction

- Val manifest discovery uses `head_val` variants, then generic `val.json[1]` .
- Synthetic val data allowed if manifest missing (for shape tests); metrics become uninformative in that case.
- If val positives are absent, AP may be NaN; training script skips best-tracking when NaN appears.

A11. Best Checkpoint Logic

- Monitor metric defaults to `mAP` ; for TTC-focused runs, switch monitor to `ttc_mae` and note `greater_is_better=False` .
- Best checkpoint payload duplicated to `trackB_best.pt` for stable downstream access.
- Summary JSON includes metrics and config snapshot; mirrored to `trackB_best_summary.json` .

A12. Evaluation Details on Semantics

- Frame-level metrics require noun/verb heads; if absent, semantic metrics are skipped.
- IoU threshold `iou_thresh` (default 0.5) gates whether top candidate counts as a hit.
- Top-5 metrics consider the top five scored candidates after priors/CLIP blending.
- Per-noun/verb stats track total/correct counts to diagnose class imbalance.

A13. Hotspot and CLIP Implementation Notes

- Hotspot default score used when (noun, verb) pair absent; blended linearly with next-active probability.
- CLIP noun prompts built from STA labels if available; fallback prompt `noun {id}` otherwise.
- CLIP similarity normalized to [0,1]; blended via `clip_weight` .
- Both priors operate on predicted noun/verb, not ground truth.

A14. Overlay Rendering

- Colors rotate through a small palette; text shows probability and TTC prediction with optional GT.
- Saved to `runs/Track_B/overlays/val/` with filename `<uid>_<frame>.jpg` .
- Top-K overlays controlled by `evaluation.topk_overlay` .

A15. Predictions File Uses

- CSV/JSONL can be ingested by Track C or external analysis notebooks.
- Fields include both base probability (`prob_pos`) and blended score (`score_final`), enabling ablations on priors.
- TTC error per candidate recorded for debugging regression quality.

A16. Pre-Extracted Tokens Practicalities

- Generation is external to this repo; ensure consistent preprocessing (img_size, normalization).
- Detection heuristic reads first `.pt` file to infer dim and backbone; ensure files follow expected key names.
- When using tokens_root, dataset skips backbone instantiation, saving memory and startup time.

A17. VideoMAE Considerations

- Input expects `time_len=16` ; mismatch leads to incorrect token shapes.
- Freeze encoder to avoid heavy fine-tuning unless resources allow; unfreezing requires LR tuning.
- tokens_root for VideoMAE reduces runtime substantially; necessary for large-scale sweeps.

A18. Dataset Debug Aids

- On first 50 parsed records, eval prints counts of empty candidates and missing paths.
- Missing UID list written to cache for quick inspection.
- Invalid records logged as JSON strings in CSV; open in spreadsheet for cleanup.

A19. Synthetic Mode Behavior

- If manifests are missing and `synthetic_if_empty=True` , each UID receives one negative candidate; used for smoke tests only.
- Metrics on synthetic data are not meaningful; use only for pipeline debugging.

A20. Temporal Window Choice and Motion Sensitivity

- Short windows (T=8) capture immediate motion cues with less compute; longer windows (T=16) capture slower interactions but cost more.
- Stride controls spacing; stride 1 is denser but heavier.
- Align with Track A frame sampling to ensure consistent timing.

A21. Candidate Ordering and Masks

- Dataset preserves manifest order after parsing; training shuffles batches, not candidates within a frame.
- Padding mask ensures losses ignore padded slots; semantics losses further mask to positives.

A22. Normalization of TTC

- `normalize_ttc=True` applies z-score using dataset mean/std; stored per dataset instance.
- Eval denormalizes predictions using stored stats for MAE computation.
- If TTC variance is near zero, std is clamped to 1.0 to avoid division by zero.

A23. Label Smoothing Impact

- Reduces overconfidence on noisy positives; default 0.05.
- Keep small to avoid blurring minority class signals when positives are scarce.

A24. Weight Decay and Regularization

- Weight decay 0.01 balances generalization; consider lowering if using frozen backbones and small heads.
- Dropout in fusion and head provides additional regularization.

A25. Training Duration and Patience

- 20 epochs typical for moderate dataset; early stopping patience 5 prevents over-training.
- For tiny datasets, reduce epochs and patience to avoid plateauing early.

A26. Batch Size Sensitivity

- Batch size influences LR scaling; current schedule does not auto-scale LR.
- For small batch sizes (<4), consider reducing LR to maintain stability.

A27. AMP Usage

- Disabled by default; can accelerate on GPU but monitor for inf/nan losses.
- Keep loss scaling via GradScaler (`enabled=cfg.amp`).

A28. Candidate Limit Strategy

- Training: limit to 16 by default to bound compute; random subsampling introduces mild stochasticity.
- Evaluation: same limit applied; if you need exhaustive scoring, raise or disable limit (set high).

A29. Manifest Discovery Order

- Train: Stage B head_train.* preferred, then manifests root.
- Val: Stage B head_val.* preferred, then val.* fallbacks.
- Stage B run discovery scans `runs/Track_A` for latest directory with head manifests.

A30. Frame Path Resolution Fallbacks

- Absolute path in manifest used if exists; else frames_root/uid plus basename; else index-based; else last frame.
- Useful when manifests were generated on different machines with different roots.

A31. Overlay and Prediction Alignment

- Overlays use post-prior/CLIP scores to rank candidates; consistent with reported metrics.
- Predictions include both base and blended scores to allow post-hoc comparisons.

A32. Metrics Stability Notes

- If positives are extremely rare, AP can be unstable; interpret alongside accuracy and TTC MAE.
- Frame-level metrics require IoU overlap; ensure boxes are reasonably aligned (Track A Stage B crops clamp boxes).

A33. Plotting and Aggregation

- `trackB_plots.py` scans metrics JSON files and aggregates into PNG + TSV; extend script if new metrics added.
- Track C comparison script (`trackC_compare_metrics.py`) can contrast Track B metrics against pruning results.

A34. Documentation Hygiene for Thesis

- When citing results, mention config name (e.g., `trackB_videomae_ego`), TTC mode, priors usage, and manifest source.
- Provide checkpoints and metrics file paths for reviewers to reproduce tables/plots.

A35. Additional Commands (Linux/Bash equivalents)

```
python local_extraction/trackB/trackB_train_loader.py --config trackB --epochs 10
python local_extraction/trackB/trackB_eval.py --ttc_mode binned --checkpoint local_extraction/runs/Track_B/checkpoints/trackB_best.pt
```

A36. Interpreting Per-Noun/Verb Stats

- `per_noun_stats` : map from noun ID to total/correct/accuracy; highlight long-tail failures.
- `per_verb_stats` : same for verbs; useful when verbs are sparse.
- Consider merging with taxonomy metadata for readable tables.

A37. Handling Missing Semantics

- If noun/verb IDs are -1 or missing, those candidates do not contribute to semantic metrics.
- Keep `use_multi_task_labels=False` if manifests lack semantics to avoid meaningless losses.

A38. TTC Bin Edges Customization

- Set `multi_task.ttc_thresholds` in YAML to adjust bins; ensure monotonic increasing.
- Bins implicitly define $N+\delta$ and All metrics; report binning scheme in thesis.

A39. Candidate Geometry and ROI Pooling

- ROI pooling operates on token grid, not raw pixels; coarse alignment may reduce sensitivity to tiny boxes.
- If candidates are extremely small, consider reducing `img_size` or using finer backbone (not provided).

A40. Using Stage B Consolidated Heads

- When manifest rows come from Stage B `head_train.jsonl` , the dataset regroups by `(uid, frame_name)` automatically.
- `is_positive` parsed from boolean or string; cls inferred accordingly.

A41. Cache Contents and Usage

- `invalid_records.csv` : CSV of raw JSON strings for records that failed parsing.
- `missing_val_uids.txt` : list of UID dirs missing under frames_root (eval debug).
- TTC stats stored in dataset instance; not persisted to disk by default.

A42. Mixing Priors and CLIP

- Priors and CLIP are independent; you can enable one or both.
- Blending order: hotspot first, then CLIP (final score used for ranking).
- Tune weights separately; avoid overweighting when base model already confident.

A43. Manifest Authoring Example with Minimal Fields

```
{
  "uid": "sample123",
  "frame": 42,
  "candidates": [
    {"x1": 10, "y1": 20, "x2": 60, "y2": 80, "is_positive": 1}
  ]
}
```

- The parser will set `cls=1`, `verb_id=-1`, `noun_id=-1`, `ttc=0.0`, `ttc_bin=0`.

A44. Semantic Manifest Example (Clip-level)

```
[
  {
    "uid": "clip001",
    "frame": 99,
    "candidates": [
      {"x1": 100, "y1": 150, "x2": 220, "y2": 260, "is_positive": 1,
    "verb_id": 7, "noun_id": 42, "ttc": 0.7}
    ]
  }
]
```

A45. Stage B Discovery Edge Cases

- If multiple Stage B runs exist with similar timestamps, ensure the intended one is latest by modification time or set `stageB_run` explicitly.
- If head manifests are missing in a Stage B run, discovery skips it.

A46. Handling Large Candidate Sets

- If Track A Stage B writes many boxes per frame, use `stage_b.keep_top_n` upstream or `candidate_limit` downstream.
- Very large candidate sets increase padding and slow fusion.

A47. Potential Extensions (not implemented)

- Distributed training: would require refactoring tokenization out of dataset.
- Hard negative mining: could be added by re-weighting negatives; not present in code.
- Dynamic proposal refinement: currently not supported; proposals are static from Track A.

A48. Sanity Scripts and Helpers

- `trackB_show_samples.py` (if present) can visualize samples.
- `trackB_show_config.py` prints resolved configs.
- `trackB_fusion.py` runnable for a quick fusion smoke test.

A49. Dataset Shapes at a Glance

- Image tokens: `(N, 512)` for ResNet18; `N ≈ 7*7` at `img_size 224`.
- Video tokens: `(T, N, 512)` with T time steps.
- Projected tokens: `(N, 256)` after projector.
- ROI pooled features per candidate: `(Nc, 256)` per modality; stacked and padded to `(B, Nc_max, 256)`.

A50. Notes on Label Noise

- Next-active labels may be noisy when proposals miss hands; consider increasing `K` in Track A to reduce false negatives.
- TTC labels may be coarse; binning can mitigate sensitivity to small errors.

A51. Interaction with Track C

- Track C pruning applies a runtime budget; compare metrics before/after pruning using the same manifests.
- Keep `trackB_best.pt` consistent across pruning sweeps to isolate pruning effects.

A52. Evaluation Time Complexity

- Dominated by tokenization if `tokens_root` absent; otherwise dominated by fusion/head on candidates.
- Overlays add image I/O overhead; disable for large-scale sweeps.

A53. Candidate Ordering in Predictions

- Predictions are stored in manifest order; scores allow downstream sorting.
- Frame-level metrics internally reorder candidates by final score for top-1/top-5 evaluation.

A54. Handling Different Frame Rates

- Temporal stride is frame-count based; if frame rate varies, consider normalizing stride to time or pre-sampling frames uniformly.

A55. Pitfalls with Tokens Root Paths

- If `tokens_root` is relative, training script resolves against `local_extraction` ; ensure directory exists.
- If auto-detection fails, projector input may be wrong; verify `Projector: <in> -> <out>` printout.

A56. Normalization Consistency Between Train and Eval

- Eval denormalizes TTC using stats from eval dataset, not from training stats saved in checkpoint.
- Ensure train/val datasets have similar TTC distributions if comparing MAE.

A57. Logging Behavior When RunLogger Absent

- If `core.RunLogger` import fails, scripts print a warning and continue; no side effects.
- This ensures portability to minimal environments.

A58. Handling Multiple Checkpoints

- `_find_latest_checkpoint` selects latest `trackB_final_*.pt` if none specified; use explicit path for reproducibility.
- Best checkpoint may not correspond to latest final; prefer `trackB_best.pt` for reporting.

A59. Impact of Label Smoothing on AP

- AP can drop slightly with heavy smoothing; keep smoothing modest (0.05 default).

A60. ROI Pool Granularity

- Since ROI pooling averages tokens, extremely thin boxes may lose detail; consider adjusting Track A boxes or using larger crop sizes upstream.

A61. Numerical Stability

- TTC normalization clamps std; prevents division by near-zero but may understate variance in tiny datasets.
- AMP can introduce inf/nan if gradients explode; disable if observed.

A62. Parallelism Considerations

- Multiprocessing DataLoader not supported due to backbone objects; would require moving tokenization out of dataset.

A63. Manifests with Absolute Image Paths

- Supported; dataset uses provided absolute path if it exists, else falls back to `frames_root` basename logic.

A64. UID Naming

- UID directory names under `frames_root` are used directly; ensure consistency with manifests to avoid missing frames.

A65. Temporal Window Sampling Edge Cases

- If window shorter than `time_len` due to start-of-video, `sample_window_ending_at` may return fewer frames; ensure code handles empties (it warns and skips).

A66. Extending Metrics

- `trackB_metrics.py` can be extended with additional metrics; remember to add to plots if needed.

A67. Configuration Override Strategy

- For reproducibility, prefer editing YAML rather than CLI overrides; include YAML diff in thesis appendix if needed.

A68. Snapshotting Experiment Settings

- Store a small README in each run directory summarizing key toggles (mode, TTC, priors, backbone) to ease comparisons.

A69. Frame-Level Positive Assumption

- Metrics assume one positive per frame; if multiple positives exist, only the first positive index is used for frame-level matching.

A70. Handling Null TTC

- If TTC missing, defaults to 0.0; consider setting to a neutral value in manifests to avoid bias.

A71. Tokenizer Mean/Std

- Defaults match ImageNet statistics; if using domain-specific backbone, adjust if needed (not exposed in YAML by default).

A72. VideoMAE Input Size

- Patch size/tubelet size set in config; ensure extracted frames/crops align with assumed size (`img_size`).

A73. Weight Initialization

- Projector and fusion use PyTorch defaults; head initialized via default `nn.Linear`; no custom init provided.

A74. Class Distribution Logging

- Validation prints class counts; inspect to detect extreme imbalance.

A75. Sanity Check After Training

- Training script evaluates one batch for preview accuracy and TTC prediction; useful quick check on checkpoint sanity.

A76. TTC Bin Prediction in Eval

- If bin head exists, predicted bin comes from logits; otherwise derived from regression outputs.

A77. Overlay Text Placement

- Text drawn slightly above box; may clip at top edge; acceptable for qualitative review.

A78. Filesystem Assumptions

- Workspace-write permissions required; sandbox must allow writes to `local_extraction/runs/Track_B/`.

A79. Large-Scale Sweeps

- Disable overlays and predictions if only metrics are needed to reduce I/O.
- Consider precomputing tokens and using a lighter fusion depth.

A80. Time-to-Contact Units

- TTC stored and evaluated in seconds.
- Ensure consistency if manifests were generated with different frame rates; TTC should already be normalized to seconds upstream.

A81. Handling Negative Coordinates

- Candidate boxes are assumed valid; Stage B clamping already applied; Track B does not clamp further.

A82. Config Inheritance

- `trackB_videomae_ego.yaml` inherits `trackB.yaml`; changes propagate unless overridden.

A83. Tokenizer State Sharing

- `set_tokenizer_config` propagates loaded YAML to tokenizer module; ensures consistent defaults across scripts.

A84. Device Selection

- TokenizerConfig sets device to CUDA if available; override by setting `runtime.device` in base config or env variable if supported.

A85. Backbone Freezing

- ResNet18 frozen by default; to finetune, modify tokenizer to unfreeze parameters (not exposed in YAML).

A86. Weight Decay Scope

- Applies to all parameters in optimizer; no parameter grouping used.

A87. Manifest Reuse Across Experiments

- Reuse Stage B manifests for multiple Track B configs to isolate model changes; note manifest timestamp in results.

A88. Frame Resolution Dependence

- ROI pooling uses image size to map to token grid; consistent resolution across frames assumed.

A89. Handling Mixed Manifests

- If manifest includes both clip and video variants, ensure only one is passed; dataset takes first matching file.

A90. Stage B Crop Size Irrelevance

- Track B operates on full frames, not cropped images; crop size in Stage B only affects saved crops, not manifest coordinates.

A91. TTC Normalization Across Splits

- Train and val TTC stats computed separately; differences can affect MAE comparability; document if significant.

A92. Multi-Class Next-Active

- If manifests include multiple class labels, head expands; metrics switch to multiclass accuracy/mAP.
- Positive definition still uses `is_positive` when present; ensure consistency to avoid confusion.

A93. Storage Footprint Estimates

- Checkpoints: few MB (small model).
- Metrics/predictions: size proportional to candidate count; overlays can be large if many frames.

A94. Latency Considerations

- Fusion/head lightweight; tokenization dominates; pre-extracted tokens or smaller `time_len` reduce latency.

A95. Potential Future Work (for thesis discussion)

- Integrate motion flow tokens; add trajectory-based priors; explore contrastive pretraining for tokens.

This chapter intentionally exceeds 1000 lines to serve as a comprehensive, thesis-ready reference for Track B. It can be trimmed for publication but preserves both theoretical framing and practical execution details as written.